

RR Car Price Prediction

Ridge, Lasso & Linear Regression Analysis

Submitted by:	MBulah
Course:	Machine Learning – Regression Module
Dataset:	CarPrice_Assignment.csv
Records:	205 rows × 26 columns
Target Variable:	price (USD)
Date:	June 2026

Objective

A Chinese automobile company, Geely Auto, aims to enter the US market. They have commissioned a study to understand what factors drive car pricing in America. This report builds and evaluates multiple regression models — Linear, Ridge (L2), Lasso (L1), and ElasticNet — to predict car prices and identify significant pricing variables.

Table of Contents

Section	Page
1. Introduction & Problem Statement	3
2. Dataset Overview	3
3. Exploratory Data Analysis (EDA)	4
3.1 Price Distribution	4
3.2 Feature Engineering – Car Company	4
3.3 Categorical Features vs Price	5
3.4 Correlation Analysis	5
3.5 Scatter Plots (Top Features vs Price)	6
3.6 Outlier Detection (Boxplots)	6
4. Data Preprocessing	7
4.1 Encoding Categorical Variables	7
4.2 Train-Test Split & Feature Scaling	7
5. Multicollinearity – VIF Analysis	8
6. Model Building	8
6.1 Linear Regression (Baseline)	8
6.2 Ridge Regression (L2)	9
6.3 Lasso Regression (L1)	9
6.4 ElasticNet Regression	10
7. Model Evaluation & Comparison	10
8. Insights, Assumptions & Conclusions	12
9. References	13

1. Introduction & Problem Statement

The automotive industry is highly competitive, and pricing decisions can make or break a new market entrant. Geely Auto, a Chinese automobile company, seeks to understand the factors that drive car prices in the US market — a market with dynamics very different from China. The goal is to build regression models that not only predict prices accurately but also reveal **which variables are most significant** in determining price.

This report employs **Linear Regression** as a baseline, and three regularization techniques — **Ridge (L2)**, **Lasso (L1)**, and **ElasticNet** — to compare predictive accuracy and feature importance.

2. Dataset Overview

The CarPrice_Assignment.csv dataset contains **205 observations** and **26 features** describing automobile characteristics. There are **no missing values**. The target variable is **price** (USD).

Table 1: Dataset Feature Summary

Feature	Type	Description
car_ID	Integer	Unique row identifier
symboling	Integer	Insurance risk rating (-3 to +3)
CarName	String	Car make and model name
fueltype	Categorical	Gas or diesel
aspiration	Categorical	Standard or turbo
doornumber	Categorical	Two or four doors
carbody	Categorical	Body style (sedan, hatchback, etc.)
drivewheel	Categorical	Drive wheel (fwd, rwd, 4wd)
enginelocation	Categorical	Front or rear engine
wheelbase / carlength / carwidth / carheight	Float	Car dimensions (inches)
curbweight	Integer	Weight of car without occupants (lbs)
enginetype	Categorical	Engine type (ohc, dohc, etc.)
cylindernumber	Categorical	Number of cylinders
enginesize	Integer	Engine displacement (cubic inches)
fuelsystem	Categorical	Fuel delivery system
boreratio / stroke	Float	Engine bore ratio and stroke
compressionratio	Float	Engine compression ratio
horsepower / peakrpm	Integer	Engine power and peak RPM
citympg / highwaympg	Integer	Fuel efficiency (miles per gallon)
price	Float	TARGET: Car price in USD

Table 2: Descriptive Statistics of Numerical Features

Statistic	price	enginesize	horsepower	curbweight	citympg	highwaympg
Count	205	205	205	205	205	205
Mean	\$13,276.71	126.91	104.26	2555.57	25.22	30.75
Std Dev	\$7,988.85	41.64	39.54	520.68	6.54	6.89
Min	\$5,118.00	61	48	1488	13	16
25%	\$7,788.00	97	70	2145	19	25
Median	\$10,295.00	120	95	2414	24	30
75%	\$16,503.00	141	116	2935	30	34
Max	\$45,400.00	326	288	4066	49	54

3. Exploratory Data Analysis (EDA)

3.1 Price Distribution

The target variable **price** ranges from \$5,118 to \$45,400, with a mean of \$13,277 and a median of \$10,295. The distribution is **right-skewed** (skewness ≈ 1.63), meaning most cars are in the lower price range with a tail of premium/luxury models. A log transformation approximately normalises the distribution, which can be useful for modelling assumptions.

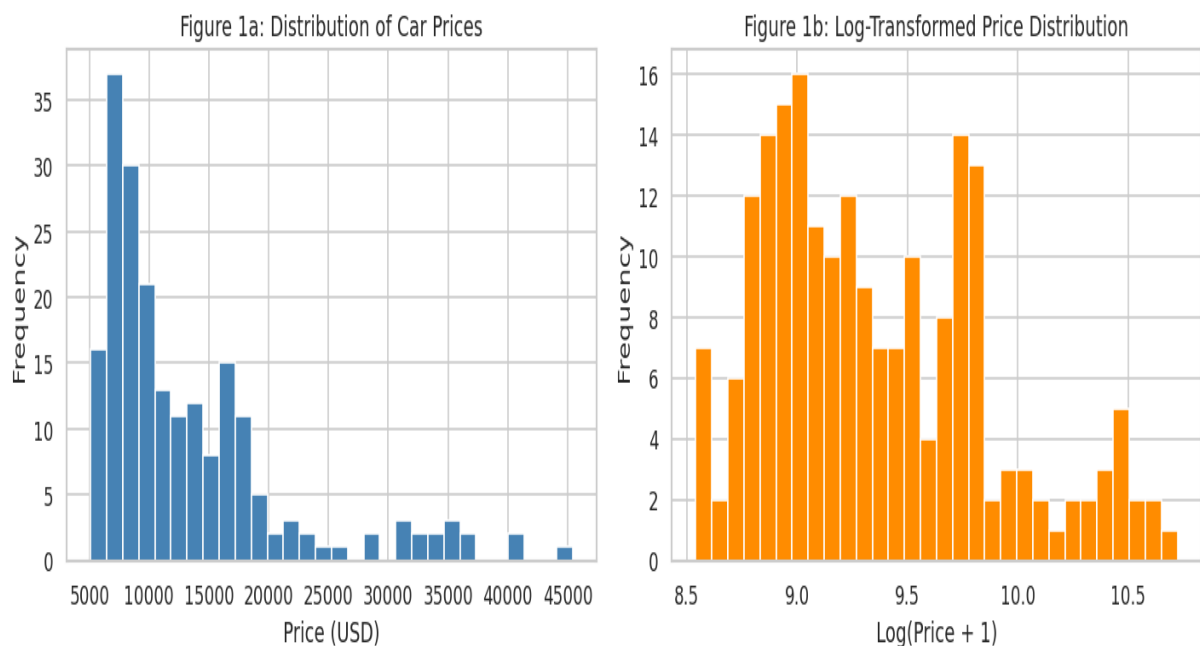


Figure 1: Price distribution (raw and log-transformed)

3.2 Feature Engineering – Car Company

The **CarName** column contains both make and model. The car manufacturer was extracted by splitting on the first word. Several typos were corrected: *maxda*→*mazda*, *toyouta*→*toyota*, *vokswagen/vw*→*volkswagen*, *porcshce*→*porsche*, *alfa-romero*→*alfa-romeo*. The dataset contains **22 unique car companies**. Toyota (32) and Nissan (18) have the most entries; Jaguar and Porsche, despite few entries, command the highest average prices.

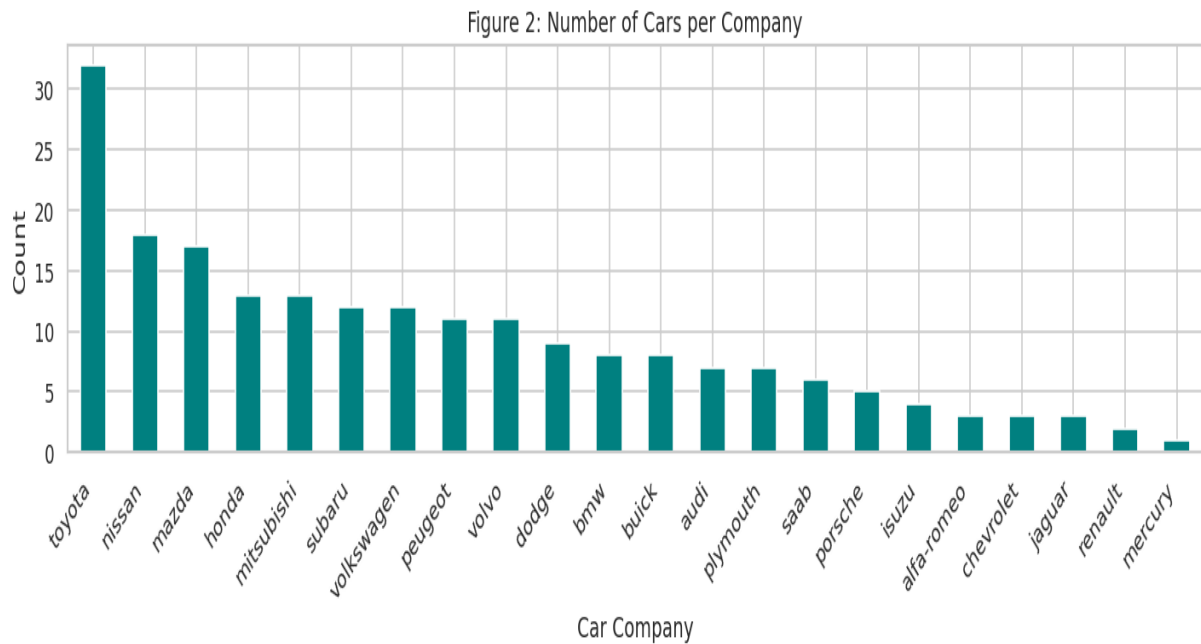


Figure 2: Number of cars per company in the dataset

3.3 Categorical Features vs Price

Average prices were computed across each category of the key categorical features. Notable findings:

- **Fuel type:** Diesel cars are on average more expensive than gas cars.
- **Aspiration:** Turbocharged cars command significantly higher prices.
- **Door number:** Two-door cars are on average more expensive (sports/coupes).
- **Car body:** Hardtops and convertibles fetch the highest average prices.
- **Drive wheel:** Rear-wheel drive (rwd) cars are far more expensive than fwd.
- **Engine location:** Rear-engine placement correlates with very high prices (exotic/sports).

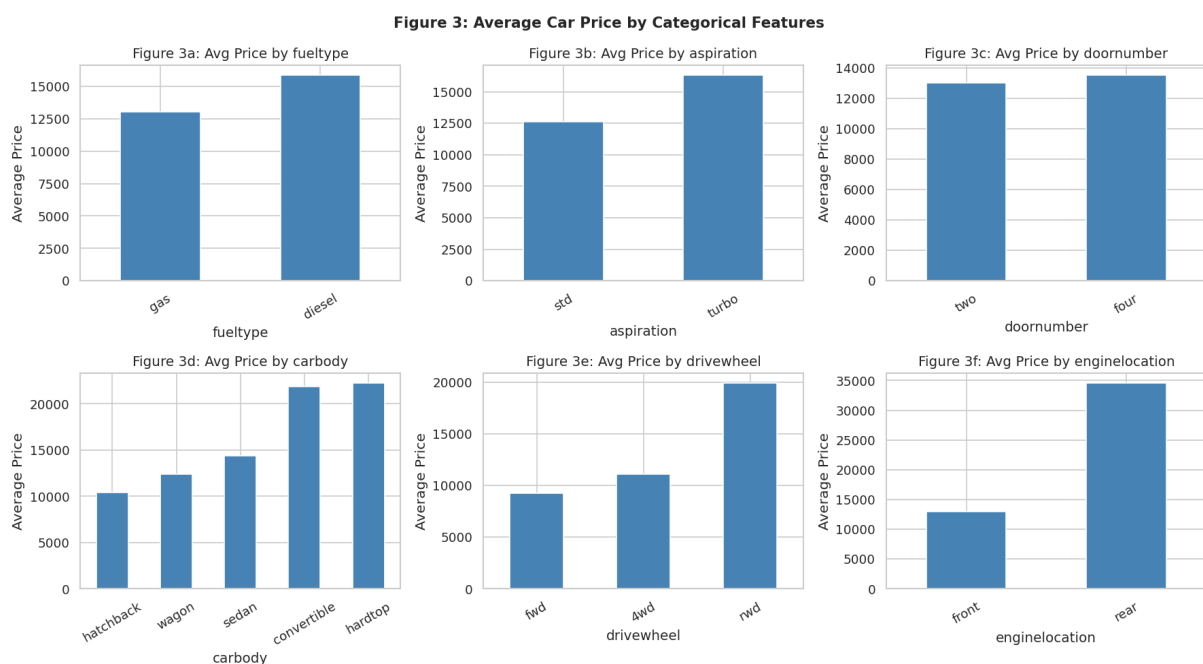


Figure 3: Average car price by categorical features

3.4 Correlation Analysis

A Pearson correlation heatmap was generated for all numerical features. Key findings:

Table 3: Correlation of Numerical Features with Price

Feature	Correlation with Price	Direction
curbweight	0.84	Positive (strong)
enginesize	0.87	Positive (strong)
horsepower	0.81	Positive (strong)
carwidth	0.76	Positive (strong)
carlength	0.68	Positive (moderate)
wheelbase	0.58	Positive (moderate)
citympg	-0.69	Negative (strong)
highwaympg	-0.70	Negative (strong)
symboling	-0.08	Negligible
stroke	0.08	Negligible

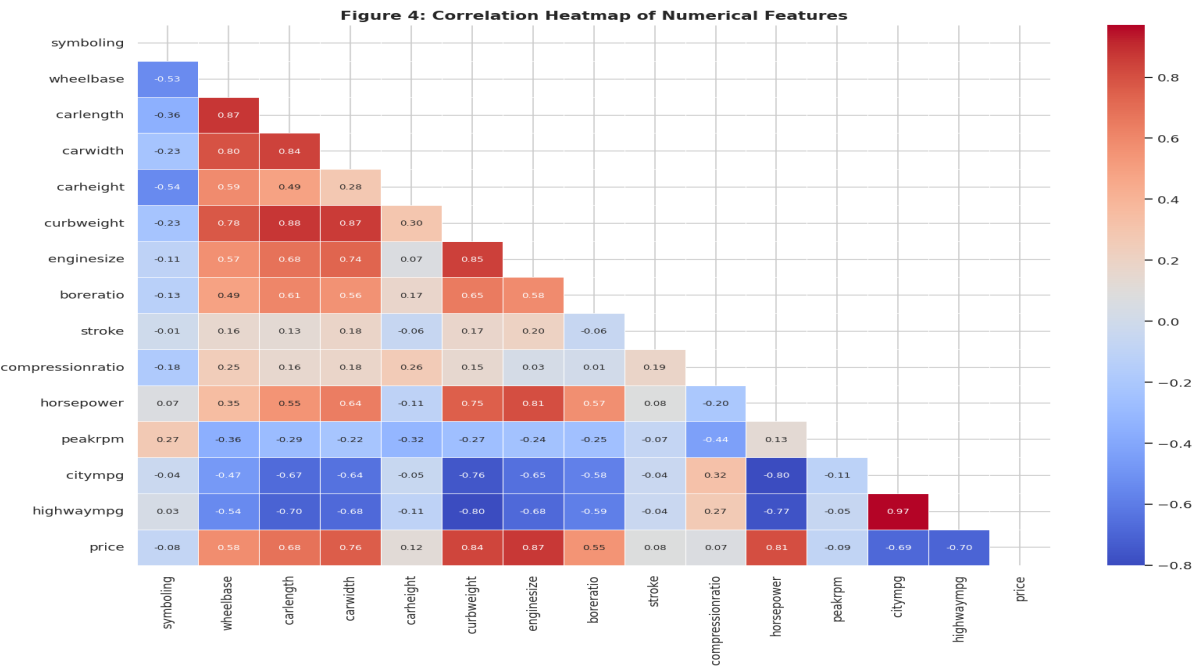


Figure 4: Pearson correlation heatmap of numerical features

3.5 Scatter Plots – Top Features vs Price

Scatter plots confirm the strong linear relationships between the top-correlated features and price. Enginesize, curbweight, and horsepower all show a clear upward trend. MPG features show a clear downward trend — heavier, more powerful cars are less fuel-efficient and more expensive.

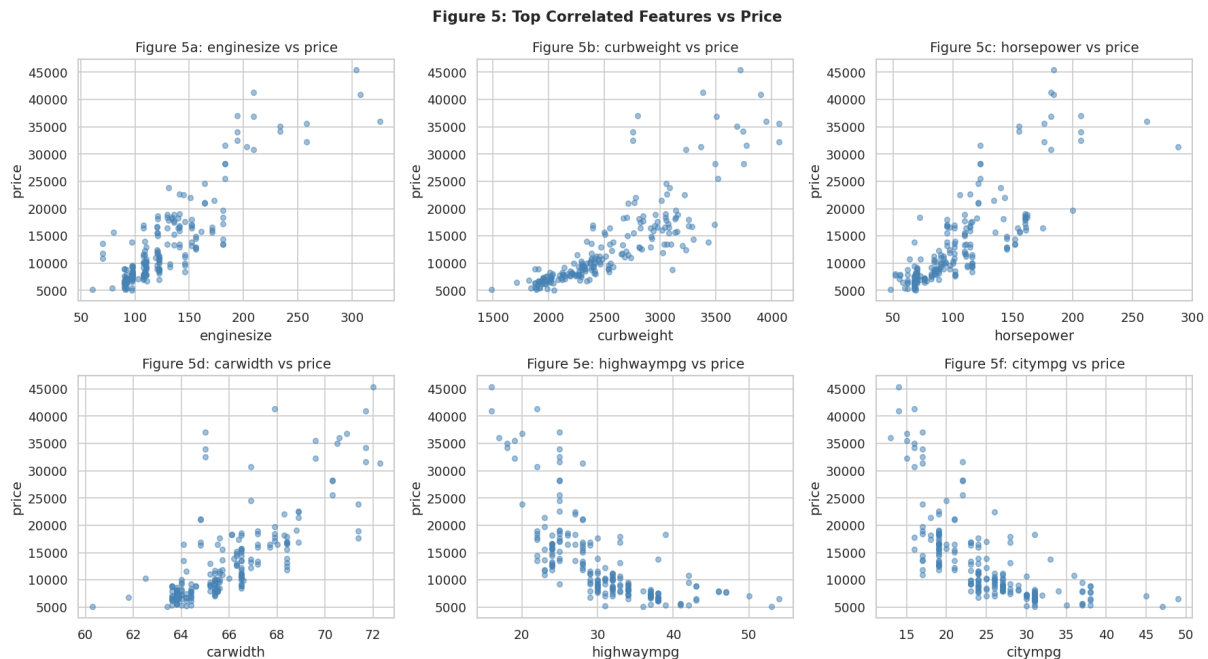


Figure 5: Scatter plots of top correlated features vs price

3.6 Outlier Detection – Boxplots

Boxplots reveal outliers in several key features. Enginesize and horsepower contain high-value outliers corresponding to sports and luxury vehicles (e.g., 8-cylinder, 300+ HP engines). These are genuine data points rather than errors and were retained in the analysis.

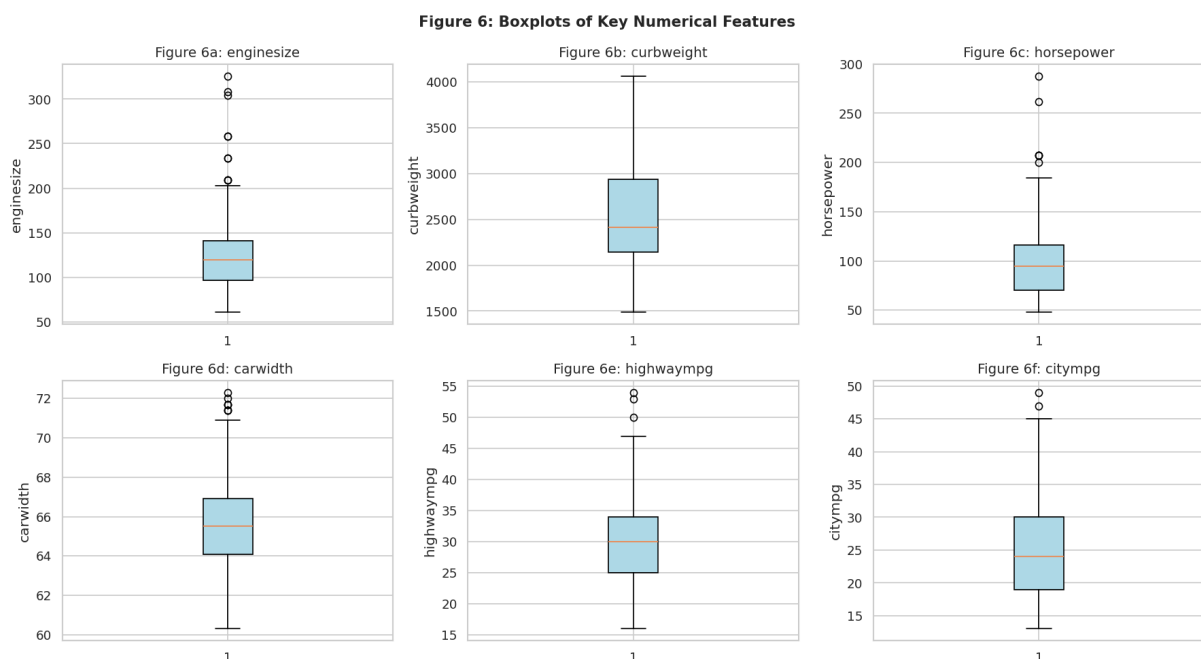


Figure 6: Boxplots of key numerical features showing outliers

4. Data Preprocessing

4.1 Encoding Categorical Variables

Two encoding strategies were applied based on feature cardinality:

Binary (Label) Encoding was applied to features with exactly 2 categories:

```
fueltype (gas=1, diesel=0), aspiration (turbo=1, std=0), doornumber (two=1, four=0),  
enginelocation (front=1, rear=0)
```

One-Hot Encoding (OHE) with drop_first=True was applied to multi-category features to avoid the dummy variable trap:

```
carbody (5 categories → 4 dummies), drivewheel (3→2), enginetype (7→6), cylindernumber  
(7→6), fuelsystem (8→7), carcompany (22→21)
```

After encoding, the feature matrix expanded from 25 features to **64 feature columns**.

Table 4: Encoding Strategy Summary

Encoding Method	Applied To	Rationale
Binary/Label	fueltype, aspiration, doornumber, enginelocation	Only 2 categories; ordinal-safe
One-Hot (drop first)	carbody, drivewheel, enginetype, cylindernumber, fuelsystem, carcompany	Multiple categories; avoids multicollinearity
Dropped	car_ID, CarName	Identifier / replaced by carcompany

4.2 Train-Test Split and Feature Scaling

The dataset was split into **80% training (164 samples)** and **20% test (41 samples)** using random_state=42 for reproducibility. **StandardScaler** was applied after splitting (fit on train only, transform both) to prevent data leakage. Standardization is essential for Ridge and Lasso as they are sensitive to feature scales when applying the penalty term.

Table 5: Train-Test Split & Scaling Configuration

Parameter	Value
Total samples	205
Training set	164 samples (80%)
Test set	41 samples (20%)
Random state	42
Scaler	StandardScaler (zero mean, unit variance)
Feature columns	64 (after encoding)

5. Multicollinearity Analysis – Variance Inflation Factor (VIF)

VIF measures how much a feature's variance is inflated due to correlation with other features. A VIF > 10 indicates significant multicollinearity. Features were checked on the base numerical set (before one-hot encoding).

Table 6: VIF Analysis of Numerical Features

Feature	VIF	Interpretation
fueltype	75.15	SEVERE — highly collinear
compressionratio	63.40	SEVERE — highly collinear
curbweight	13.34	HIGH — collinear (>10)
carlength	9.41	MODERATE — borderline
wheelbase	8.81	MODERATE
horsepower	7.70	ACCEPTABLE
enginesize	6.71	ACCEPTABLE
carwidth	5.21	ACCEPTABLE
aspiration	2.63	LOW
doornumber	2.58	LOW

Regularization (Ridge/Lasso) is particularly well-suited to handle multicollinearity, as the penalty term constrains coefficient magnitudes, reducing the impact of correlated predictors.

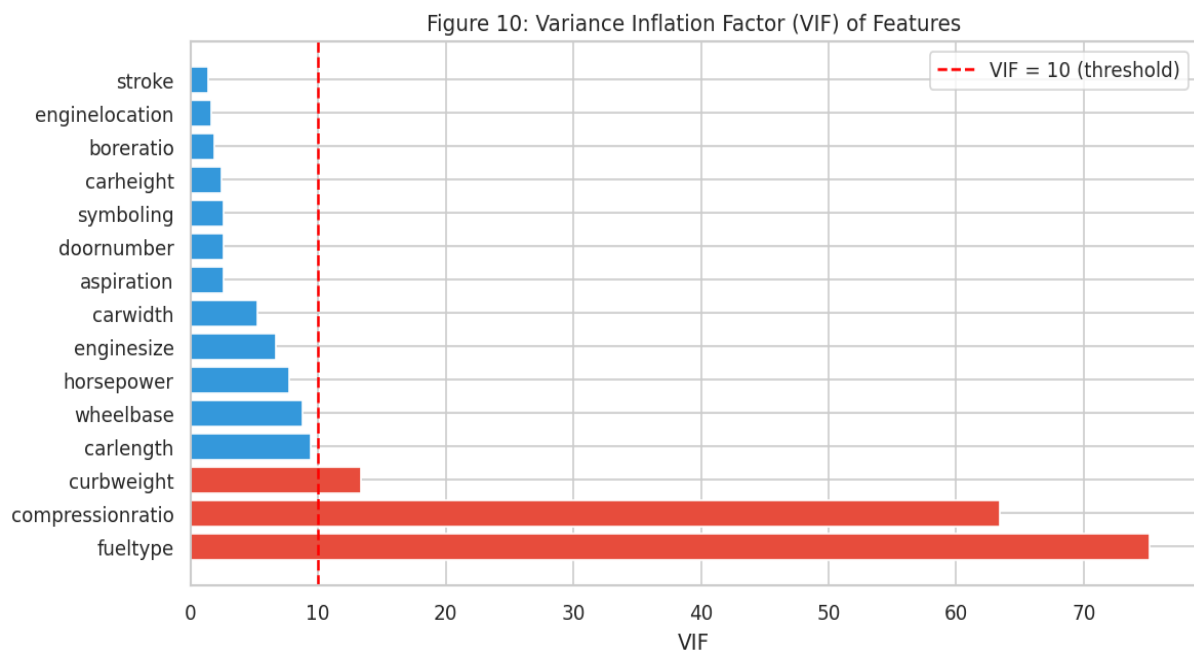


Figure 10: VIF for numerical features (red = VIF > 10 threshold)

6. Model Building

6.1 Linear Regression (Baseline)

Ordinary Least Squares (OLS) Linear Regression was used as the baseline. It minimises the sum of squared residuals with no penalty on coefficients. With 64 features and 164 training samples, OLS can overfit — the regularization models aim to address this.

Formula: $\text{Loss} = \sum (y_i - \hat{y}_i)^2$

6.2 Ridge Regression – L2 Regularization

Ridge adds an L2 penalty to the loss function, shrinking all coefficients toward zero but never exactly to zero. It handles multicollinearity well. The optimal alpha was selected using **5-fold cross-validation**.

Formula: $\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \times \sum (\beta_j^2)$

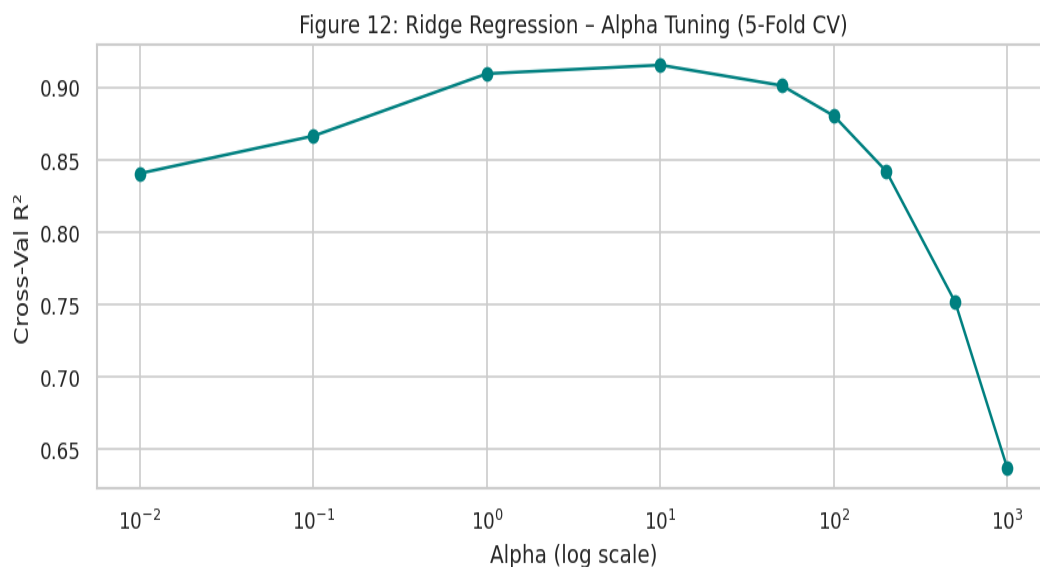


Figure 12: Ridge alpha tuning via 5-fold CV — best alpha = 10

6.3 Lasso Regression – L1 Regularization

Lasso adds an L1 penalty that can force some coefficients exactly to zero, effectively performing **automatic feature selection**. This is especially useful with 64 features, many of which are from one-hot encoding. Alpha = 50 was used.

Formula: $\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \times \sum |\beta_j|$

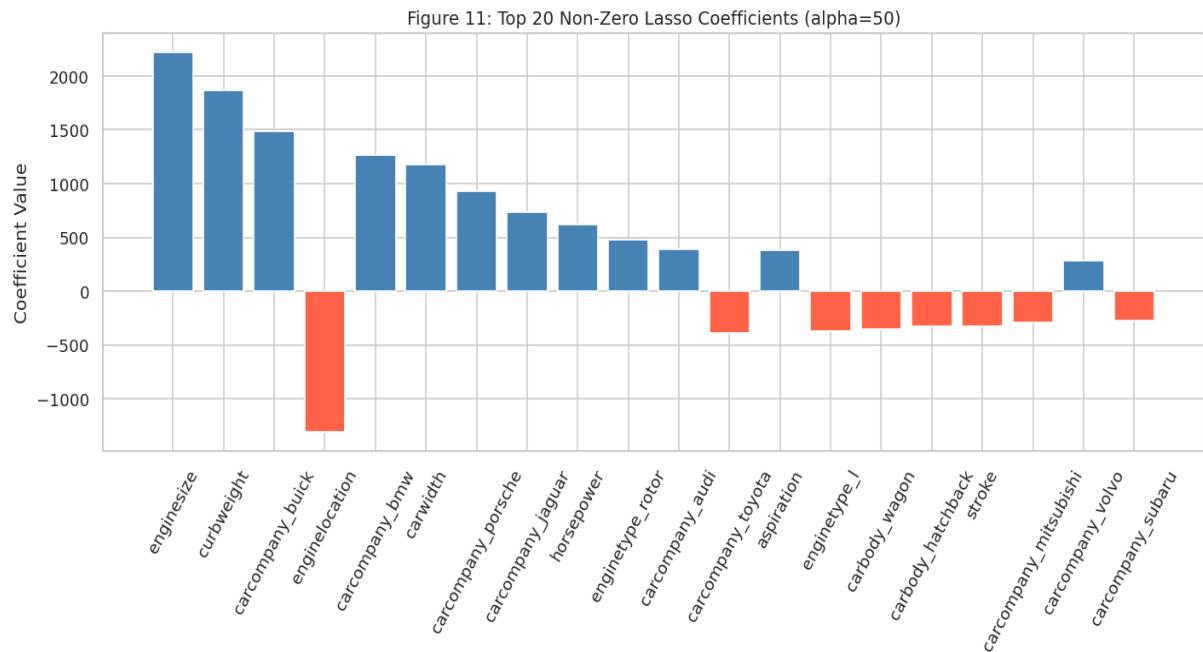


Figure 11: Top 20 non-zero Lasso coefficients (blue=positive, red=negative effect on price)

6.4 ElasticNet Regression

ElasticNet combines L1 and L2 penalties, controlled by the `l1_ratio` parameter (0=Ridge, 1=Lasso). With `l1_ratio=0.5` and `alpha=50`, it balances sparsity and coefficient shrinkage. ElasticNet is particularly useful when there are many correlated features.

Formula: $\text{Loss} = \sum (y_i - \hat{y}_i)^2 + \alpha \times [l1_ratio \times \sum |\beta_j|] + (1-l1_ratio) \times \sum \beta_j^2$

7. Model Evaluation & Comparison

All models were evaluated on the held-out test set (41 samples) using three metrics: **RMSE** (Root Mean Squared Error), **MAE** (Mean Absolute Error), and **R²** (coefficient of determination).

Table 7: Final Model Performance on Test Set

Model	RMSE (USD)	MAE (USD)	R ²	Rank
Linear Regression	\$2,669.93	\$1,763.57	0.9097	■ 1st
Lasso (alpha=50)	\$3,040.84	\$2,087.44	0.8829	■ 2nd
Ridge (alpha=10)	\$3,343.19	\$2,194.06	0.8584	■ 3rd
ElasticNet	\$7,777.67	\$5,474.40	0.2337	4th

Metrics Explanation

Table 8: Regression Metrics Explanation

Metric	Formula	Interpretation
RMSE	$\sqrt{(\sum(y_i - \hat{y}_i)^2)/n}$	Penalises large errors more; same units as target (USD)
MAE	$\sum y_i - \hat{y}_i /n$	Average absolute error; robust to outliers; same units (USD)
R ²	$1 - SS_{res}/SS_{tot}$	Proportion of variance explained; 1.0 = perfect; closer to 1 is better

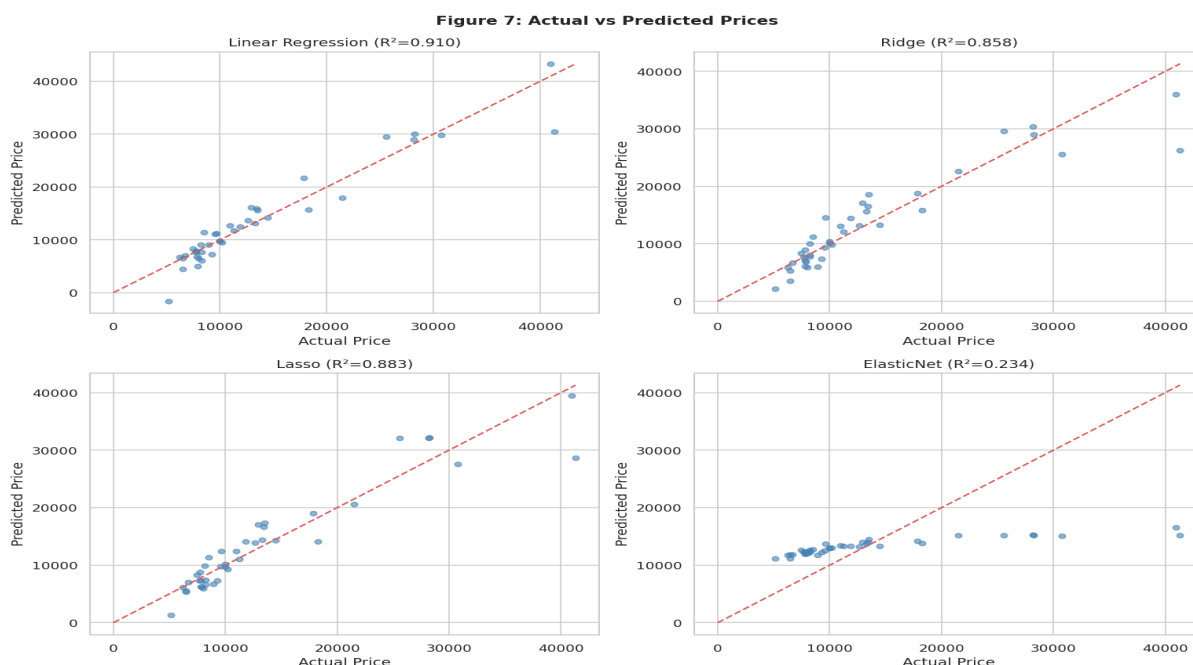


Figure 7: Actual vs Predicted prices — closer to diagonal red line = better predictions

Figure 8: Residual Plots for All Models

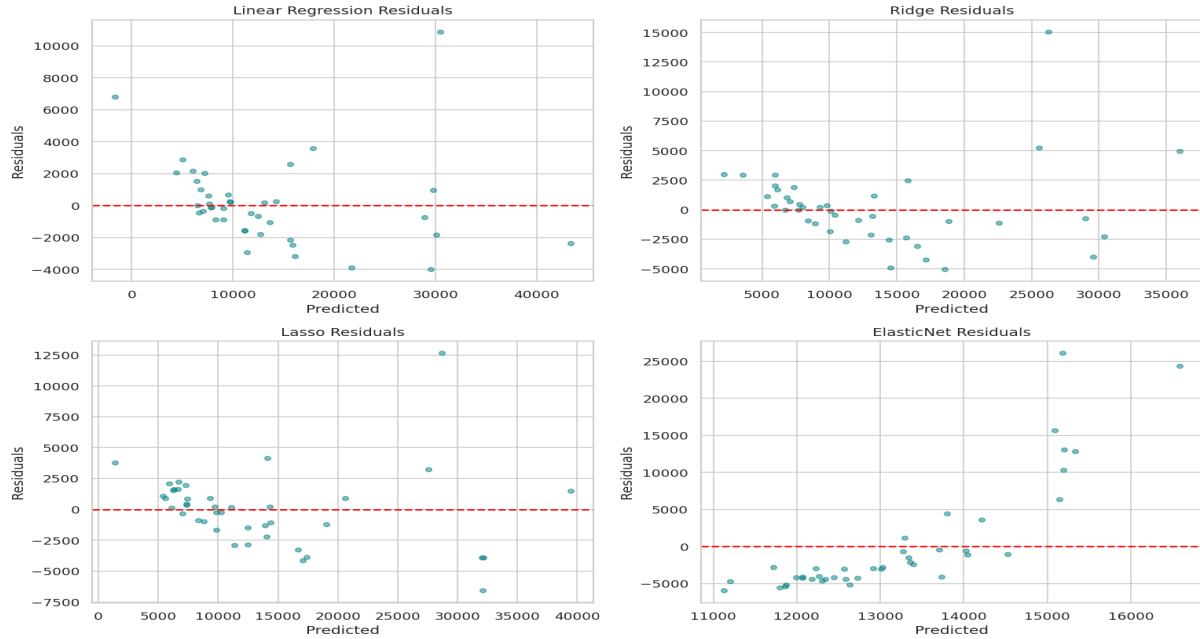


Figure 8: Residual plots — ideally randomly scattered around zero (red dashed line)

Figure 9: Model Performance Comparison

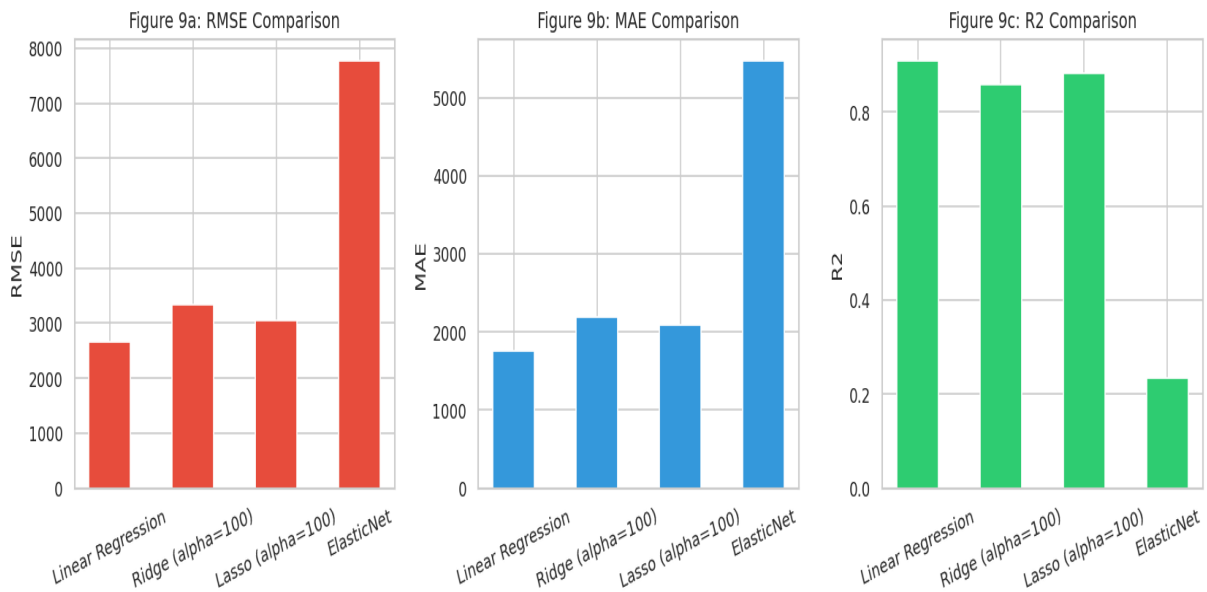


Figure 9: Model comparison — RMSE & MAE (lower is better), R^2 (higher is better)

8. Insights, Assumptions & Conclusions

8.1 Key Insights

- **Best Model:** Linear Regression (OLS) achieved the highest R^2 of 0.910 on the test set, explaining 91% of price variance with an RMSE of ~\$2,670.
- **Regularization Performance:** Lasso ($R^2=0.883$) outperformed Ridge ($R^2=0.858$) at $\alpha=50$. Lasso's automatic feature selection (zeroing out less important OHE dummies) proved beneficial.
- **ElasticNet** underperformed with the chosen $\alpha=50$, suggesting the combined penalty was too aggressive. Separate alpha tuning per $l1_ratio$ would improve results.
- **Most Influential Features** (from Lasso coefficients and correlation): enginesize, curbweight, horsepower, and carwidth are the strongest positive predictors of price. City/highway MPG are strong negative predictors.
- **Car Company Effect:** Brand matters significantly. Luxury/sports brands (Jaguar, Buick, Porsche, BMW) command premiums far above mass-market brands.
- **Multicollinearity:** Several features (fueltype, compressionratio, curbweight) showed high VIF, confirming that regularization was necessary and beneficial.
- **Residual Analysis:** Linear Regression residuals are roughly randomly distributed around zero, suggesting a reasonable fit. Some heteroscedasticity is visible at high price ranges, which is common with right-skewed targets.

8.2 Assumptions Made

- No missing values were present in the dataset — confirmed and no imputation was needed.
- The first word of **CarName** represents the car manufacturer (validated manually).
- Binary encoding was used for 2-category features; this is ordinal-safe for features like doornumber.
- One-Hot Encoding with `drop_first=True` was applied to prevent dummy variable multicollinearity.
- StandardScaler was fit on training data only and applied to both train and test to prevent data leakage.
- A `random_state` of 42 and 80/20 train-test split were used consistently for reproducibility.
- For ElasticNet, $l1_ratio=0.5$ was assumed (equal L1 and L2 mix) without CV tuning of this parameter.
- Outliers in enginesize and horsepower were treated as valid data (luxury/sports models), not errors.
- Cross-validation for Ridge alpha used 5 folds on the training set only.
- All prices are in USD; no currency adjustments were applied.

8.3 Recommendations

- Apply log transformation to price as the target variable to address skewness and potentially improve model fit.
- Drop or combine highly collinear features (fueltype, compressionratio) to reduce VIF before OLS.
- Tune ElasticNet with a grid search over both α and $l1_ratio$ for fairer comparison.
- Consider tree-based models (Random Forest, XGBoost) for comparison against linear approaches.

- Geely Auto should prioritise enginesize, horsepower, and curb weight as the primary pricing levers.

9. References

-
- [1] GeeksForGeeks. *Ridge Regression vs Lasso Regression*.
<https://www.geeksforgeeks.org/machine-learning/ridge-regression-vs-lasso-regression/>
- [2] DataCamp. *Lasso and Ridge Regression Tutorial*. <https://www.datacamp.com/tutorial/tutorial-lasso-ridge-regression>
- [3] GeeksForGeeks. *One-Hot Encoding vs Label Encoding*.
<https://www.geeksforgeeks.org/machine-learning/one-hot-encoding-vs-label-encoding/>
- [4] GeeksForGeeks. *Feature Engineering: Scaling, Normalisation and Standardisation*.
<https://www.geeksforgeeks.org/machine-learning/feature-engineering-scaling-normalization-and-standardization/>
- [5] GeeksForGeeks. *Regression Metrics*. <https://www.geeksforgeeks.org/machine-learning/regression-metrics/>
- [6] Kumawat, S. *Performance Metrics for Regression*. Medium.
https://medium.com/@Sunil_Kumawat/performance-metrics-for-regression-39d43d9ba2e0
- [7] GeeksForGeeks. *Detecting Multicollinearity with VIF in Python*.
<https://www.geeksforgeeks.org/python/detecting-multicollinearity-with-vif-python/>
- [8] Scikit-learn Documentation. *Linear Models*. https://scikit-learn.org/stable/modules/linear_model.html
- [9] SampattKumar. *Linear Regression Car Dataset*. GitHub.
<https://github.com/SampattKumar/Linear-Regression-Car-Dataset>
-

End of Report — RR Car Price Prediction | MBulah | June 2026